

□ Nav2 Integration Report — Multi-Robot Autonomous Frontier Exploration

Project: `multi_robot_exploration` (Project 08 — RoboticsIITJ) **Platform:** ROS 2 Humble · Nav2 · Gazebo Classic · TurtleBot3 Burger **Date:** October 2026

Table of Contents

1. Executive Summary
 2. System Architecture Overview
 3. Nav2 Components Used (Per Robot)
 4. Complete ROS 2 Topic Map
 5. Topic Publishing Rates
 6. TF Frame Tree
 7. Nav2 Configuration Deep-Dive
 8. System Collaboration — How Everything Fits Together
 9. Custom Nodes Interfacing with Nav2
 10. Launch Sequence & Lifecycle Management
 11. Multi-Robot Namespacing Strategy
 12. Known Issues & Design Decisions
 13. Key Presentation Talking Points
-

1. Executive Summary

This project deploys **two TurtleBot3 Burger robots** in a procedurally-generated Gazebo world to **autonomously explore unknown environments** using coordinated frontier-based exploration. **Nav2** is the core navigation framework that enables each robot to plan paths, avoid obstacles, and execute motion commands.

What Nav2 Does in This Project

Capability	Nav2 Component	Role
Path Planning	Theta Star Planner	Computes any-angle optimal paths on the global costmap
Path Following	DWB Local Planner	Generates velocity commands to track the planned path
Obstacle Avoidance	Costmap 2D (local + global)	Maintains real-time obstacle maps from LiDAR data
Recovery Behaviors	Behavior Server (Spin/BackUp/Wait)	Automatically recovers from stuck/blocked states
Velocity Smoothing	Velocity Smoother	Applies jerk limits for smooth, safe motion
Goal Management	BT Navigator	Orchestrates plan→execute→recover via Behavior Trees
Waypoint Following	Waypoint Follower	Sequential multi-goal navigation for demos

What is Custom (Built Around Nav2)

Capability	Custom Node	How it Uses Nav2
Frontier Detection	<code>frontier_coordinator</code>	Processes merged map, sends <code>NavigateToPose</code> action goals
Map Merging	<code>map_merge_node</code>	Fuses per-robot SLAM maps → publishes <code>/map</code> consumed by Nav2's static layer
Collision Avoidance	<code>frontier_coordinator</code>	Cancels Nav2 goals & publishes zero-vel <code>cmd_vel</code> when robots are too close
Waypoint Demo	<code>waypoint_navigator</code>	Uses Nav2's <code>NavigateToPose</code> action client for sequential waypoint driving

2. System Architecture Overview

The system operates as a **5-layer pipeline**, where each layer depends on the output of the one above it:

flowchart TB

```
subgraph L1["Layer 1 - Simulation (Gazebo)"]
    GZ["Gazebo Classic"]
    WG["generate_random_world.py"]
    SP["spawn_two_turtlebots.launch.py"]
    WG -->|SDF world file| GZ
    SP -->|Patched robot SDFs| GZ
end
```

end

```
subgraph L2["Layer 2 - Multi-Robot SLAM"]
    ST1["slam_toolbox (robot1)"]
    ST2["slam_toolbox (robot2)"]
end
```

end

```
subgraph L3["Layer 3 - Map Merging"]
    MM["map_merge_node (custom)"]
end
```

end

```
subgraph L4["Layer 4 - Nav2 Stack (x2 robots)"]
    N1["Nav2 Stack (robot1)"]
    N2["Nav2 Stack (robot2)"]
end
```

end

```
subgraph L5["Layer 5 - Exploration Intelligence"]
    FC["frontier_coordinator (custom)"]
end
```

end

GZ -->|"/robotN/scan"| L2

GZ -->|"TF: odom → base_footprint"| L4

ST1 -->|"/robot1/map"| MM

ST2 -->|"/robot2/map"| MM

MM -->|"/map (merged)"| L4

MM -->|"/map (merged)"| FC

MM -->|"Static TF: map → robotN/map"| L4

FC -->|"NavigateToPose action"| L4

N1 -->|"/robot1/cmd_vel"| GZ

N2 -->|"/robot2/cmd_vel"| GZ

[!IMPORTANT] Nav2 sits at **Layer 4** — the execution layer. It receives goals from the Frontier Coordinator (Layer 5) and translates them into velocity commands that drive the simulated robots (Layer 1). Every navigation decision — path planning, obstacle avoidance, recovery — happens inside Nav2.

3. Nav2 Components Used (Per Robot)

Each robot runs **7 Nav2 lifecycle nodes** + 1 lifecycle manager = **8 Nav2 processes per robot, 16 total**.

#	Nav2 Node	Package	Purpose
1	<code>controller_server</code>	<code>nav2_controller</code>	Runs the DWB local planner at 20 Hz to follow the global path
2	<code>planner_server</code>	<code>nav2_planner</code>	Runs Theta Star global planner to compute paths on the costmap
3	<code>bt_navigator</code>	<code>nav2_bt_navigator</code>	Orchestrates <code>NavigateToPose</code> / <code>NavigateThroughPoses</code> using Behavior Trees
4	<code>behavior_server</code>	<code>nav2_behaviors</code>	Provides Spin, BackUp, Wait recovery behaviors at 10 Hz
5	<code>smoother_server</code>	<code>nav2_smoother</code>	Path smoothing (default config)
6	<code>velocity_smoother</code>	<code>nav2_velocity_smoother</code>	Smooths <code>cmd_vel_nav</code> → <code>cmd_vel</code> at 20 Hz with jerk limits
7	<code>waypoint_follower</code>	<code>nav2_waypoint_follower</code>	Sequential waypoint following with optional pause at each waypoint
8	<code>lifecycle_manager</code>	<code>nav2_lifecycle_manager</code>	Auto-starts all 7 nodes above via <code>autostart: true</code>

Managed Lifecycle Node Order

```
LIFECYCLE_NODE_NAMES = [  
    "controller_server",  
    "smoother_server",  
    "planner_server",  
    "behavior_server",  
    "bt_navigator",  
    "waypoint_follower",  
    "velocity_smoother",  
]
```

The lifecycle manager transitions these in order: `unconfigured → inactive → active`. With `bond_timeout: 0.0`, it doesn't enforce heartbeat bonds — this prevents spurious node-death false alarms in simulation.

4. Complete ROS 2 Topic Map

4.1 Topics Published/Subscribed Per Robot

Each `robotN` namespace contains the following topics (shown for `robot1`; `robot2` is identical with namespace substitution):

Sensor Topics (Gazebo → Nav2/SLAM)

Topic	Message Type	Publisher	Subscribers	Rate
<code>/robot1/scan</code>	<code>sensor_msgs/LaserScan</code>	Gazebo LiDAR plugin	<i>slamtoolbox</i> , <i>localcostmap</i> (<i>obstaclelayer</i>), <i>globalcostmap</i> (<i>obstacle_layer</i>)	~5 Hz
<code>/robot1/odom</code>	<code>nav_msgs/Odometry</code>	Gazebo diff_drive plugin	Nav2 (via TF)	~30 Hz
<code>/robot1/imu</code>	<code>sensor_msgs/Imu</code>	Gazebo IMU plugin	(not directly used by Nav2)	~30 Hz

SLAM Topics

Topic	Message Type	Publisher	Subscribers	Rate
<code>/robot1/map</code>	<code>nav_msgs/OccupancyGrid</code>	<i>slam_toolbox</i> (<i>robot1</i>)	<i>mapmergenode</i>	~5 Hz (RELIABLE, TRANSIENT_LOCAL)

Merged Map (Global)

Topic	Message Type	Publisher	Subscribers	Rate
<code>/map</code>	<code>nav_msgs/OccupancyGrid</code>	<i>mapmergenode</i>	<i>frontiercoordinator</i> , <i>globalcostmap</i> (<i>static_layer</i>) of both robots	2 Hz (RELIABLE, TRANSIENT_LOCAL)

Nav2 Internal Topics (Per Robot)

Topic	Message Type	Publisher	Subscriber	Rate
/robot1/cmd_vel_nav	geometry_msgs/Twist	controller_server	velocity_smoother	20 Hz
/robot1/cmd_vel	geometry_msgs/Twist	velocitysmoother (or frontiercoordinator emergency)	Gazebo diff_drive	20 Hz
/robot1/local_costmap/costmap	nav_msgs/OccupancyGrid	local_costmap	controller_server	2 Hz
/robot1/local_costmap/costmap_raw	nav2_msgs/Costmap	local_costmap	behavior_server	2 Hz
/robot1/local_costmap/published_footprint	geometry_msgs/PolygonStamped	local_costmap	behavior_server	2 Hz
/robot1/global_costmap/costmap	nav_msgs/OccupancyGrid	global_costmap	plannerserver, explorelite	1 Hz
/robot1/global_costmap/costmap_updates	map_msgs/OccupancyGridUpdate	global_costmap	explore_lite	1 Hz
/robot1/plan	nav_msgs/Path	planner_server	controller_server	On-demand

Nav2 Action Servers (Per Robot)

Action	Action Type	Server Node	Client(s)
/robot1/navigate_to_pose	nav2_msgs/NavigateToPose	bt_navigator	frontiercoordinator, waypointnavigator
/robot1/navigate_through_poses	nav2_msgs/NavigateThroughPoses	bt_navigator	(available, not currently used)
/robot1/follow_waypoints	nav2_msgs/FollowWaypoints	waypoint_follower	(available, not currently used)
/robot1/compute_path_to_pose	nav2_msgs/ComputePathToPose	planner_server	bt_navigator
/robot1/follow_path	nav2_msgs/FollowPath	controller_server	bt_navigator
/robot1/spin	nav2_msgs/Spin	behavior_server	bt_navigator (recovery)
/robot1/backup	nav2_msgs/BackUp	behavior_server	bt_navigator (recovery)
/robot1/wait	nav2_msgs/Wait	behavior_server	bt_navigator (recovery)

Visualization Topics (Custom)

Topic	Message Type	Publisher	Subscriber	Rate
/frontiers_marker s	visualization_msgs/MarkerArray	frontier_coordinator	RViz2	Every 3s (replan)
/frontier_goals_markers	visualization_msgs/MarkerArray	frontier_coordinator	RViz2	Every 3s (replan)

5. Topic Publishing Rates

Configured Rates Summary

System	Parameter	Rate	Source File
Controller Server	controller_frequency	20 Hz	nav2paramsrobot1.yaml
Local Costmap	update_frequency	5 Hz	nav2paramsrobot1.yaml
Local Costmap	publish_frequency	2 Hz	nav2paramsrobot1.yaml
Global Costmap	update_frequency	1 Hz	nav2paramsrobot1.yaml
Global Costmap	publish_frequency	1 Hz	nav2paramsrobot1.yaml
Planner Server	expected_planner_frequency	20 Hz (max)	nav2paramsrobot1.yaml
Behavior Server	cycle_frequency	10 Hz	nav2paramsrobot1.yaml
Velocity Smoother	smoothing_frequency	20 Hz	nav2paramsrobot1.yaml
Waypoint Follower	loop_rate	20 Hz	nav2paramsrobot1.yaml
Map Merge	publish_rate	2 Hz	mapmergeparams.yaml
Frontier Coordinator	replan timer	0.33 Hz (every 3s)	frontiercoordinator.py
Frontier Coordinator	safety loop	10 Hz	frontiercoordinator.py
Explore Lite (alt)	planner_frequency	0.33 Hz (every 3s)	exploremulti.launch.py

TF Transform Rates (from recorded `view_frames`)

Transform	Publisher	Rate
<code>map</code> → <code>robot1/map</code>	mapmergenode (Static TF)	Once (latched)
<code>map</code> → <code>robot2/map</code>	mapmergenode (Static TF)	Once (latched)
<code>robot1/map</code> → <code>robot1/odom</code>	slam_toolbox	~ 5.4 Hz
<code>robot2/map</code> → <code>robot2/odom</code>	slam_toolbox	~ 5.4 Hz
<code>robot1/odom</code> → <code>robot1/base_footprint</code>	Gazebo diff_drive	~ 30 Hz
<code>robot2/odom</code> → <code>robot2/base_footprint</code>	Gazebo diff_drive	~ 30 Hz
<code>robotN/base_footprint</code> → <code>robotN/base_link</code>	robotstatepublisher	Static
<code>robotN/base_link</code> → <code>robotN/base_scan</code>	robotstatepublisher	Static
<code>robotN/base_link</code> → <code>robotN/wheel_*_link</code>	robotstatepublisher	~ 20 Hz

6. TF Frame Tree

The complete TF tree with all 14 frames for the 2-robot system:

```

graph TD
    MAP["map<br/>(global merged frame)"]

    R1M["robot1/map"]
    R2M["robot2/map"]
    R1O["robot1/odom"]
    R2O["robot2/odom"]
    R1BF["robot1/base_footprint"]
    R2BF["robot2/base_footprint"]
    R1BL["robot1/base_link"]
    R2BL["robot2/base_link"]
    R1BS["robot1/base_scan"]
    R2BS["robot2/base_scan"]
    R1IMU["robot1/imu_link"]
    R2IMU["robot2/imu_link"]
    R1WL["robot1/wheel_left_link"]
    R1WR["robot1/wheel_right_link"]
    R2WL["robot2/wheel_left_link"]
    R2WR["robot2/wheel_right_link"]
    R1CB["robot1/caster_back_link"]
    R2CB["robot2/caster_back_link"]

    MAP -->|"Static TF (map_merge_node)"| R1M
    MAP -->|"Static TF (map_merge_node)"| R2M
    R1M -->|"slam_toolbox ~5.4Hz"| R1O
    R2M -->|"slam_toolbox ~5.4Hz"| R2O
    R1O -->|"diff_drive ~30Hz"| R1BF
    R2O -->|"diff_drive ~30Hz"| R2BF
    R1BF -->|"Static"| R1BL
    R2BF -->|"Static"| R2BL
    R1BL -->|"Static"| R1BS
    R1BL -->|"Static"| R1IMU
    R1BL -->|"Static"| R1CB
    R1BL -->|"~20Hz"| R1WL
    R1BL -->|"~20Hz"| R1WR
    R2BL -->|"Static"| R2BS
    R2BL -->|"Static"| R2IMU
    R2BL -->|"Static"| R2CB
    R2BL -->|"~20Hz"| R2WL
    R2BL -->|"~20Hz"| R2WR

```

Key TF Chain for Nav2

The critical TF chain that Nav2 requires for each robot is:

```
map → robotN/map → robotN/odom → robotN/base_footprint → robotN/base_link → robotN/base_scan
```

Segment	Publisher	Why Nav2 Needs It
map → robotN/map	map_merge_node (Static TF)	Unifies both robots into a shared global frame
robotN/map → robotN/odom	slam_toolbox	Corrects odometry drift (SLAM localization)
robotN/odom → robotN/base_footprint	Gazebo diff_drive	Real-time odometry for local planning
robotN/base_footprint → robotN/base_link	robot_state_publisher	Robot model kinematics
robotN/base_link → robotN/base_scan	robot_state_publisher	Transforms LiDAR data to robot frame

7. Nav2 Configuration Deep-Dive

7.1 Global Planner — Theta Star

```
GridBased:
  plugin: "nav2_theta_star_planner/ThetaStarPlanner"
  how_many_corners: 8          # 8-connected grid (diagonal movement)
  w_euc_dist: 1.0              # Euclidean distance heuristic weight
  allow_unknown: true          # Can plan through unknown space
```

[!TIP] Why Theta Star? Unlike A, Theta Star produces **any-angle paths** that don't stair-step along grid cells. This is critical for exploration where robots navigate through previously unknown space. The `allow_unknown: true` setting is essential — without it, the planner would refuse to path into uncharted territory.*

7.2 Local Planner — DWB (Dynamic Window Approach, Backwards-compatible)

Parameter	Value	Significance
<code>max_vel_x</code>	0.15 m/s	TurtleBot3 Burger safe max linear speed
<code>max_vel_theta</code>	1.0 rad/s	Maximum rotational speed
<code>max_speed_xy</code>	0.15 m/s	Speed cap for safety
<code>sim_time</code>	2.5 s	How far ahead trajectories are simulated
<code>vx_samples</code>	20	Number of linear velocity samples
<code>vtheta_samples</code>	20	Number of angular velocity samples

DWB Critics & Their Weights

The local planner uses a weighted sum of trajectory critics to score candidate trajectories:

Critic	Scale	Purpose
<code>RotateToGoal</code>	32.0 (highest)	Ensures the robot faces the goal at arrival
<code>GoalDist</code>	24.0	Minimizes distance to the goal pose
<code>PathAlign</code>	24.0	Keeps the robot aligned with the global path direction
<code>GoalAlign</code>	20.0	Aligns the robot heading toward the goal
<code>PathDist</code>	16.0	Minimizes deviation from the global path
<code>ObstacleFootprint</code>	0.1	Penalizes proximity to obstacles (low to avoid over-conservatism)
<code>Oscillation</code>	(default)	Detects and penalizes back-and-forth oscillation

[!NOTE] The `ObstacleFootprint.scale: 0.1` is intentionally low. At near-lethal costmap values (253), the cost becomes $253 \times 0.1 = 25.3$, which is competitive with `PathDist`/`GoalDist`. This prevents the robot from being overly obstacle-avoidant in tight spaces while still keeping it safe.

7.3 Costmap Configuration

Local Costmap

Parameter	Value	Notes
<code>rolling_window</code>	<code>true</code>	Ego-centric, follows the robot
<code>width × height</code>	3m × 3m	Immediate surroundings only
<code>resolution</code>	0.05 m/pixel	5 cm per grid cell
<code>robot_radius</code>	0.105 m	TurtleBot3 Burger circular footprint
<code>update_frequency</code>	5 Hz	How fast new LiDAR data is integrated
<code>publish_frequency</code>	2 Hz	How fast the costmap is published
Layers	<code>obstaclelayer</code> , <code>inflationlayer</code>	No static_layer (intentional for rolling window)

The obstacle layer subscribes to `/robotN/scan` with:

- `raytrace_max_range`: 3.0m (clears cells along laser ray)
- `obstacle_max_range`: 2.5m (marks cells as obstacles)
- Both `clearing: True` and `marking: True` — this treats the other robot as a **dynamic obstacle** that gets cleared from the costmap within seconds of moving away

Global Costmap

Parameter	Value	Notes
<code>rolling_window</code>	<code>false</code> (default)	Fixed, map-aligned grid
<code>resolution</code>	0.05 m/pixel	Matches SLAM resolution
<code>track_unknown_space</code>	<code>true</code>	Essential for frontier exploration
<code>update_frequency</code>	1 Hz	Slower than local (less critical)
Layers	<code>staticlayer</code> , <code>obstaclelayer</code> , <code>inflation_layer</code>	Full stack including static map

The `static_layer` subscribes to `/map` (the merged map) with `map_subscribe_transient_local: True`, which means it picks up the latest merged map even if it was published before the costmap started.

Inflation Layer

Parameter	Value	Notes
<code>cost_scaling_factor</code>	8.0	How sharply costs decay from obstacles (higher = sharper falloff)
<code>inflation_radius</code>	0.15–0.25 m	Inscribed radius padding around obstacles

7.4 Behavior Server — Recovery Behaviors

Behavior	Plugin	When Triggered
<code>spin</code>	<code>nav2_behaviors/Spin</code>	Robot is stuck; rotates in place to clear obstacles
<code>backup</code>	<code>nav2_behaviors/BackUp</code>	Robot can't proceed; backs up along its trajectory
<code>wait</code>	<code>nav2_behaviors/Wait</code>	Temporary block; waits for obstacle to move

```
global_frame: robot1/odom      # Recovery behaviors operate in odom frame
robot_base_frame: robot1/base_footprint
max_rotational_vel: 1.0        # rad/s during spin
min_rotational_vel: 0.4        # rad/s minimum to overcome friction
simulate_ahead_time: 2.0      # seconds of forward simulation for safety
```

7.5 Velocity Smoother

```
smoothing_frequency: 20.0 Hz
feedback: "OPEN_LOOP"         # No closed-loop velocity feedback
max_velocity: [0.15, 0.0, 0.5] # [linear_x, linear_y, angular_z]
min_velocity: [-0.15, 0.0, -0.5]
max_accel: [1.0, 0.0, 1.5]
max_decel: [-1.0, 0.0, -1.5]
```

Velocity flow: `controller_server` → `cmd_vel_nav` → `velocity_smoother` → `cmd_vel` → Gazebo

7.6 BT Navigator

```
bt_loop_duration: 10          # ms per BT tick
default_server_timeout: 20     # seconds to wait for action servers
navigators: ["navigate_to_pose", "navigate_through_poses"]
```

Uses the default Nav2 behavior tree which orchestrates: **Plan** → **Smooth** → **Execute** → **(Recover if stuck)** → **Replan**.

7.7 Goal Checker

```
general_goal_checker:  
  stateful: True  
  plugin: "nav2_controller::SimpleGoalChecker"  
  xy_goal_tolerance: 0.25 m    # Position tolerance  
  yaw_goal_tolerance: 0.25 rad # Heading tolerance (~14°)
```

7.8 Progress Checker

```
progress_checker:  
  plugin: "nav2_controller::SimpleProgressChecker"  
  required_movement_radius: 0.5 m    # Must move this far...  
  movement_time_allowance: 10.0 s    # ...within this time window
```

If the robot hasn't moved 0.5m in 10 seconds, Nav2 declares it stuck and triggers recovery behaviors.

8. System Collaboration — How Everything Fits Together

8.1 The Complete Data Flow (One Navigation Cycle)

sequenceDiagram

participant FC as Frontier Coordinator

participant BT as BT Navigator

participant PS as Planner Server

participant CS as Controller Server

participant VS as Velocity Smoother

participant GZ as Gazebo

participant SLAM as slam_toolbox

participant MM as Map Merge Node

Note over GZ: Robot drives, LiDAR scans environment

GZ->>SLAM: /robotN/scan (LaserScan)

GZ->>SLAM: TF: odom → base_footprint (~30Hz)

SLAM->>MM: /robotN/map (OccupancyGrid)

SLAM->>SLAM: TF: robotN/map → robotN/odom (~5Hz)

MM->>FC: /map (merged OccupancyGrid, 2Hz)

MM->>PS: /map → static_layer (global costmap)

MM->>MM: Static TF: map → robotN/map

Note over FC: Replan timer fires (every 3s)

FC->>FC: Detect frontiers (morphological ops)

FC->>FC: Deduplicate centroids (1.5m radius)

FC->>FC: Build cost matrix (dist + separation + region)

FC->>FC: Assign frontier to robot (greedy argmin)

FC->>BT: NavigateToPose action goal

BT->>PS: ComputePathToPose action

PS->>PS: Theta Star path planning on global costmap

PS-->>BT: Path (nav_msgs/Path)

BT->>CS: FollowPath action

CS->>CS: DWB samples 20×20 trajectories at 20Hz

CS->>VS: cmd_vel_nav (Twist)

VS->>GZ: cmd_vel (smoothed Twist, 20Hz)

Note over FC: Safety loop checks (10Hz)

FC->>FC: Check inter-robot distance

alt Robots too close (<0.6m)

FC->>BT: Cancel NavigateToPose

```
FC->>GZ: Zero cmd_vel (emergency stop)
end
```

8.2 Subsystem Interaction Summary

Subsystem A	→	Subsystem B	Connection Type	Data
Gazebo	→	SLAM	Topic + TF	<code>/robotN/scan</code> , odom→base_footprint TF
SLAM	→	Map Merge	Topic + TF	<code>/robotN/map</code> , robotN/map→odom TF
Map Merge	→	Nav2 (global costmap)	Topic + TF	<code>/map</code> , map→robotN/map static TF
Map Merge	→	Frontier Coord.	Topic	<code>/map</code> (merged OccupancyGrid)
Frontier Coord.	→	Nav2 (BT Navigator)	Action	<code>NavigateToPose</code> goal/cancel
Nav2 (BT Navigator)	→	Nav2 (Planner)	Action	<code>ComputePathToPose</code>
Nav2 (BT Navigator)	→	Nav2 (Controller)	Action	<code>FollowPath</code>
Nav2 (BT Navigator)	→	Nav2 (Behaviors)	Action	<code>Spin</code> / <code>BackUp</code> / <code>Wait</code> (recovery)
Nav2 (Controller)	→	Nav2 (Vel Smoother)	Topic	<code>cmd_vel_nav</code> → <code>cmd_vel</code>
Nav2 (Vel Smoother)	→	Gazebo	Topic	<code>/robotN/cmd_vel</code>
Frontier Coord.	→	Gazebo	Topic	<code>/robotN/cmd_vel</code> (emergency stop only)
Frontier Coord.	→	RViz2	Topic	<code>/frontiers_markers</code> , <code>/frontier_goals_markers</code>

8.3 Key Collaboration Patterns

Pattern 1: Nav2 as a "Navigation Service"

The frontier coordinator treats Nav2 as a **black-box navigation service** via the `NavigateToPose` action interface. It sends a goal pose and receives callbacks for acceptance, rejection, success, or failure — without needing to know how Nav2 plans or executes the path.

Pattern 2: Shared Merged Map

Both robots' Nav2 stacks subscribe to the **same** `/map` **topic** (from `mapmergenode`) for their global costmap's static layer. This means both robots have full visibility of the combined explored area, even if they explored different parts of the world.

Pattern 3: Dual `cmd_vel` Authority

Two sources can publish to `/robotN/cmd_vel`:

- 1. **Nav2's `velocity_smoother`** (normal operation, 20 Hz)
- 2. **Frontier coordinator** (emergency stop only, zero velocity)

The coordinator only publishes when cancelling goals for collision avoidance, and immediately ceases after sending the stop command.

Pattern 4: TF as the Universal Coordinate System

All components — SLAM, Nav2, `mapmerge`, `frontier coordinator` — *communicate spatial information through the TF tree*. The `mapmerge_node`'s static transforms (`map → robotN/map`) are the critical bridge that unifies both robots' coordinate frames.

9. Custom Nodes Interfacing with Nav2

9.1 Frontier Coordinator (`frontier_coordinator.py`)

The **primary Nav2 client**. This is the "brain" of the system.

Nav2 Interfaces Used:

Interface	Type	Purpose
<code>{robot}/navigate_to_pose</code>	Action Client (<code>NavigateToPose</code>)	Send exploration goals
<code>{robot}/cmd_vel</code>	Publisher (<code>Twist</code>)	Emergency stop (zero velocity)
TF2 Lookup	Buffer/Listener	Get robot positions in <code>map</code> frame

How It Uses Nav2:

- 1. **Goal Sending:** Creates `NavigateToPose.Goal` with a `PoseStamped` in the `map` frame and sends it via `ActionClient.send_goal_async()`
- 2. **Goal Tracking:** Registers `_goal_response_callback` (acceptance/rejection) and `_goal_result_callback` (success/failure/cancel)
- 3. **Goal Cancellation:** Calls `goal_handle.cancel_goal_async()` when:
 - Robot is stuck (3 replan ticks with <0.1m movement)
 - Frontier disappeared from the map

- Inter-robot collision risk detected
- A better frontier is assigned

4. **Failure Handling:** When Nav2 rejects a goal or fails navigation, the frontier is blacklisted for 30 seconds to prevent infinite retry loops

State Machine:

```

IDLE → (frontier assigned) → NAVIGATING → (goal reached/failed/stuck) → IDLE
                                           → (collision risk) → PAUSED → (timer) → IDLE
  
```

9.2 Map Merge Node (`map_merge_node.py`)

Feeds Nav2 with the global map.

Interface	Type	Purpose
<code>/map</code>	Publisher (<code>OccupancyGrid</code>)	Provides the merged map for Nav2's global costmap
Static TF (<code>map → robotN/map</code>)	<code>StaticTransformBroadcaster</code>	Unifies robot frames into global <code>map</code> frame

Without this node, Nav2 would only see each robot's local SLAM map, making coordinated multi-robot navigation impossible.

9.3 Waypoint Navigator (`waypoint_navigator.py`)

Demo-mode Nav2 client. Sends a sequence of predefined waypoints to each robot.

Interface	Type	Purpose
<code>/robot/navigate_to_pose</code>	Action Client	Send sequential waypoints

Uses the same `NavigateToPose` action as the frontier coordinator, with callback chains for sequential execution.

10. Launch Sequence & Lifecycle Management

10.1 Required Launch Order

```
Terminal 1: spawn_two_turtlebots.launch.py    ← Gazebo + robots
Terminal 2: multi_robot_slam.launch.py        ← SLAM (×2)
Terminal 3: map_merge.launch.py               ← Map merging
Terminal 4: nav2_bringup_multi.launch.py      ← Nav2 stacks (×2)
Terminal 5: frontier_exploration.launch.py    ← Frontier coordinator
```

10.2 Nav2 Lifecycle Autostart

The Nav2 lifecycle manager is configured for **fully automatic startup**:

```
autostart: True          # Don't require manual lifecycle transitions
bond_timeout: 0.0        # Disable node health bonds (avoids false deaths)
```

10.3 Robot2 Staggered Launch

Robot2's Nav2 stack is delayed by **8 seconds** to avoid resource contention:

```
if name == "robot2":
    ld.add_action(TimerAction(period=8.0, actions=[group]))
```

This was a fix for a resolved issue where simultaneous Nav2 startup caused failures.

10.4 Frontier Coordinator Startup Synchronization

The coordinator explicitly waits for **all** Nav2 action servers before beginning:

```
for name in self.robot_names:
    while not self.nav_clients[name].wait_for_server(timeout_sec=2.0):
        self.get_logger().info(f'...waiting for {name}/navigate_to_pose')
```

And then waits for valid TF positions for all robots before assigning any frontiers (the `all_robots_ready` flag).

11. Multi-Robot Namespacing Strategy

11.1 Namespace Isolation

Each robot's entire stack (Gazebo plugins, SLAM, Nav2) runs under a ROS namespace: `/robot1/` and `/robot2/`. This is achieved via `PushRosNamespace` in launch files.

11.2 What Gets Namespaced

Component	Namespaced?	Example Topic
Gazebo diff_drive	☐	<code>/robot1/odom</code> , <code>/robot1/cmd_vel</code>
Gazebo LiDAR	☐	<code>/robot1/scan</code>
slam_toolbox	☐	<code>/robot1/map</code>
Nav2 servers	☐	<code>/robot1/navigate_to_pose</code>
Nav2 costmaps	☐	<code>/robot1/global_costmap/costmap</code>
TF frames	☐ (via SDF patching)	<code>robot1/base_footprint</code>
mapmergenode	☐ (global)	<code>/map</code>
frontier_coordinator	☐ (global)	Subscribes to <code>/map</code> , sends to <code>/robotN/navigate_to_pose</code>

11.3 SDF Patching for TF Consistency

A critical detail: the TurtleBot3 SDF model is **patched at launch time** to inject namespaced frame names:

```
for tag in root.iter("odometry_frame"):
    tag.text = f"{name}/odom"          # robot1/odom
for tag in root.iter("robot_base_frame"):
    tag.text = f"{name}/base_footprint" # robot1/base_footprint
for tag in root.iter("frame_name"):
    tag.text = f"{name}/base_scan"     # robot1/base_scan
```

This ensures the Gazebo plugins publish TF frames that exactly match what `robot_state_publisher` (with `frame_prefix`) and Nav2 (with `robot_base_frame` config) expect.

12. Known Issues & Design Decisions

Active Issues

Issue	Severity	Impact on Nav2
Ghost obstacles from cross-robot LiDAR	Medium	Other robot appears as obstacle in costmap; mitigated by obstacle clearing and morphological filtering
Collision avoidance deadlocks	Medium	Pausing one robot can block the corridor; mitigated by spatial partitioning that prevents crossing paths
2-robot hardcoded limit	Low	Nav2 config files are per-robot; extending requires parameterized generation
No map persistence	Low	Merged map lost on shutdown; no <code>map_saver_server</code> integration

Key Design Decisions

Decision	Rationale
Theta Star over NavFn/A*	Any-angle paths avoid stair-stepping; <code>allow_unknown: true</code> essential for exploration
DWB over MPPI/RPP	Well-tested, lightweight; critics provide fine-grained trajectory scoring
Low ObstacleFootprint weight (0.1)	Prevents over-conservative behavior in tight spaces while maintaining safety
Open-loop velocity smoother	Gazebo doesn't provide reliable velocity feedback; open-loop is more stable
Bond timeout 0.0	Simulation environments have timing jitter; disabling bonds prevents false node-death alarms
Merged map on <code>/map</code> for both robots	Both robots see the full explored area, enabling better global planning
Action client (not topic) for goals	Provides acceptance/rejection feedback, result callbacks, and cancellation — essential for coordinated multi-robot systems

13. Key Presentation Talking Points

□ Slide: "Why Nav2?"

- Nav2 is the **standard navigation framework** for ROS 2 — production-grade, well-tested, modular
- Provides **plug-and-play** path planning, obstacle avoidance, and recovery behaviors
- Supports **lifecycle management** for reliable startup/shutdown

- **Action-based API** enables clean integration with custom exploration logic

📄 Slide: "Architecture"

- **5-layer pipeline:** Simulation → SLAM → Map Merging → Nav2 → Frontier Exploration
- Each layer is **independently launchable and testable**
- Nav2 is treated as a **navigation service** — the frontier coordinator doesn't care HOW paths are planned, only that they are executed

📄 Slide: "The Navigation Loop"

1. Frontier coordinator detects unexplored boundaries in the merged map
2. Assigns frontiers to robots using cost-based spatial partitioning
3. Sends `NavigateToPose` goals to each robot's Nav2 stack
4. Nav2 plans a path (Theta Star), executes it (DWB), and handles recoveries (Spin/BackUp)
5. Robot drives, LiDAR updates SLAM, new map data triggers next replan cycle

📄 Slide: "Multi-Robot Challenges Solved"

- **Namespace isolation:** Each robot runs its own Nav2 stack under `/robotN/`
- **Shared global map:** Custom `mapmergenode` fuses individual SLAM maps into `/map`
- **Coordinated exploration:** Spatial partitioning prevents robots from heading to the same area
- **Collision avoidance:** 10 Hz safety loop + Nav2's obstacle layer treats the other robot as dynamic obstacle

📄 Slide: "By the Numbers"

- **16 Nav2 nodes** (8 per robot) running simultaneously
- **20 Hz** controller frequency for responsive path following
- **2 Hz** merged map publishing for real-time costmap updates
- **3 second** replan cycle for frontier reassignment
- **0.25 m** goal tolerance for "reached" detection
- **0.05 m** costmap resolution (5 cm per grid cell)

📄 Slide: "Procedural World Generation"

- Random worlds with 8–15 obstacles (boxes + cylinders)
- Seed-based reproducibility for testing
- Automatic spawn pose validation (clearance from obstacles + inter-robot separation)
- Ensures Nav2 encounters diverse, challenging environments every run